FEMM Electrostatics File Formats

Input (`.fee`) and Output (`.res`) Specifications

D. Meeker

September 13, 2026

Contents

1 Overview

The FEMM electrostatics solver (`belasolv`) reads a problem definition from a `.fee` file and writes the solution to a `.res` file. The `.res` file begins with a verbatim copy of the input `.fee` file, followed by a `[Solution]` section containing the finite element mesh and computed voltage values.

All floating-point values are written with 17 significant digits (`%.17g` format). Fields within a line are separated by tabs or spaces. Section headers use square brackets (`[Tag]`); property block delimiters use angle brackets (`<BeginBlock>` / `<EndBlock>`). String comparisons during parsing are case-insensitive.

2 The `.fee` Input File

2.1 Global Problem Parameters

Each parameter occupies one line in the format `[Tag] = value`. All are optional; defaults are applied if missing.

Tag	Type	Default	Description
`[Format]`	int	1	File format version
`[Precision]`	double	1e-8	Solver convergence tolerance
`[MinAngle]`	double	30	Minimum mesh element angle (degrees)
`[DoSmartMesh]`	int	1	Adaptive meshing (0=off, 1=on)
`[Depth]`	double	1.0	Extrusion depth (planar problems)
`[LengthUnits]`	string	millimeters	inches, millimeters, centimeters, meters, mils, microns
`[ProblemType]`	string	planar	planar or axisymmetric
`[Coordinates]`	string	cartesian	cartesian or polar
`[Comment]`	string	(empty)	Problem description (quoted; `\n` = newline)

2.1.1 Axisymmetric External Region

Written only when `ProblemType` is axisymmetric and both `extRo` and `extRi` are nonzero:

Tag	Type	Description
`[extZo]`	double	Outer Z coordinate
`[extRo]`	double	Outer radius
`[extRi]`	double	Inner radius

2.2 Point (Nodal) Properties

```
[PointProps] = <count>
<BeginPoint>
  <PointName> = "name"
  <Vp> = 0      prescribed voltage (V)
  <qp> = 0      point charge (C/m depth)
```

```
<EndPoint>
... repeated count times
```

2.3 Boundary Properties

```
[BdryProps] = <count>
<BeginBdry>
  <BdryName> = "name"
  <BdryType> = 0
  <Vs> = 0        fixed voltage (V)
  <qs> = 0        surface charge density (C/m^2)
  <c0> = 0        mixed BC coefficient
  <c1> = 0        mixed BC coefficient
<EndBdry>
```

Boundary types:

- 0 = fixed voltage (Dirichlet): $V = V_s$
- 1 = mixed (Robin): $\hat{n} \cdot D = -c_0(V - c_1)$
- 2 = surface charge density (Neumann): applies q_s
- 3 = periodic
- 4 = antiperiodic

2.4 Material (Block) Properties

```
[BlockProps] = <count>
<BeginBlock>
  <BlockName> = "name"
  <ex> = 1        relative permittivity, x-direction
  <ey> = 1        relative permittivity, y-direction
  <qv> = 0        volume charge density (C/m^3)
<EndBlock>
```

2.5 Conductor Properties

The electrostatics solver uses *conductors* (analogous to *circuits* in the magnetics solver) to enforce equipotential constraints or prescribe total charge on electrode surfaces.

```
[ConductorProps] = <count>
<BeginConductor>
  <ConductorName> = "name"
  <Vc> = 0         prescribed voltage (V)
  <qc> = 0         prescribed total charge (C)
  <ConductorType> = 0
<EndConductor>
```

Conductor types:

- 0 = prescribed total charge ($q = q_c$; solver computes V)
- 1 = prescribed voltage ($V = V_c$; solver computes q)

2.6 Geometry

2.6.1 Nodes

```
[NumPoints] = <count>
x    y    propIdx    inGroup    conductorIdx    [lfs]
```

- `x`, `y`: coordinates (double)
- `propIdx`: index into point property list (1-based; 0 = none)
- `inGroup`: group number (int)
- `conductorIdx`: index into conductor list (1-based; 0 = none)
- `lfs`: local feature size constraint (double, optional; -1 = auto)

2.6.2 Line Segments

```
[NumSegments] = <count>
n0    n1    maxSideLength    propIdx    hidden    inGroup    conductorIdx
```

- `n0`, `n1`: endpoint node indices (0-based)
- `maxSideLength`: mesh size constraint (double; -1 = auto)
- `propIdx`: boundary property index (1-based; 0 = none)
- `hidden`: 0 = visible, 1 = suppressed from mesh
- `inGroup`: group number
- `conductorIdx`: conductor index (1-based; 0 = none)

2.6.3 Arc Segments

```
[NumArcSegments] = <count>
n0  n1  arcAngle maxSideLength propIdx hidden inGroup conductorIdx mySideLength
```

- `n0`, `n1`: endpoint node indices (0-based)
- `arcAngle`: arc span (degrees)
- `maxSideLength`: mesh constraint (double)
- `propIdx`: boundary property index (1-based; 0 = none)
- `hidden`: 0 = visible, 1 = suppressed
- `inGroup`: group number
- `conductorIdx`: conductor index (1-based; 0 = none)
- `mySideLength`: computed side length (0 = auto)

2.6.4 Holes

```
[NumHoles] = <count>
x    y    inGroup
```

Holes are seed points for regions excluded from the mesh. They correspond to block labels with material `"<No Mesh>"`.

2.6.5 Block Labels

```
[NumBlockLabels] = <count>
x    y    blockIdx    meshSize    inGroup    extFlags
```

- `x`, `y`: label position (double)
- `blockIdx`: material index (1-based; 0 = unassigned)
- `meshSize`: element size (double; ≤ 0 = auto). Stored as diameter; converted to area: $A = \pi d^2/4$
- `inGroup`: group number
- `extFlags`: bit 0 = external region, bit 1 = default region

2.7 Differences from Magnetics (`.fem`) Format

- No `[Frequency]` or `[ACSolver]` fields (electrostatics is always DC).
- No `[PrevType]` or `[PrevSoln]` fields (no incremental/frozen permeability).
- Material properties are permittivity (ε_x, ε_y) and volume charge (q_v), not permeability, coercivity, conductivity, or B-H curves.
- Point properties use voltage V_p and point charge q_p, not current and vector potential.
- *Conductors* replace *circuits*; geometry entities (nodes, segments, arcs) carry a conductor index.
- Block labels have no circuit index, turns, or magnetization direction fields.

3 The `.res` Output File

The `.res` file begins with a verbatim copy of the input `.fee` file (all sections described in Section 2), followed by the solver's solution data.

3.1 Solution Header

Immediately after the echoed `.fee` content:

```
[Solution]
```

3.2 Mesh Nodes

```
<NumNodes>
```

followed by `NumNodes` lines:

```
x    y    V    Q
```

- `x`, `y`: node coordinates in model units
- `V`: computed voltage (V)
- `Q`: boundary/conductor marker (int):
  - -2 = interior node (no boundary condition)

- -1 = fixed voltage boundary
- $0 \ldots N-1$ = conductor index

Node indices are 0-based throughout the solution section.

3.3 Mesh Elements

```
<NumElements>
```

followed by `NumElements` lines:

```
p0   p1   p2   labelIdx
```

- `p0`, `p1`, `p2`: vertex node indices (0-based)
- `labelIdx`: block label index (0-based, into the block label list from the `.fee` header)

3.4 Conductor Solution Data

```
<NumConductors>
```

followed by `NumConductors` lines:

```
V   q
```

- `V`: voltage on the conductor (V). For prescribed-voltage conductors, this is the prescribed value; for prescribed-charge conductors, this is the computed value.
- `q`: total charge on the conductor (C). For prescribed-charge conductors, this is the prescribed value; for prescribed-voltage conductors, this is the computed value.

4 Index Conventions

- **Property indices** (materials, boundaries, conductors, point properties) are **1-based** in the file. An index of 0 means "no property assigned." The parser converts to 0-based internally.
- **Node indices** (in segments, arcs, and the solution section) are **0-based**.
- **Block label indices** in the solution element list are **0-based**, referencing the block label array from the `.fee` header.

5 Unit Conversion

The `[LengthUnits]` tag determines the coordinate system. All coordinates in the file are in the specified units. Conversion to meters:

Unit	Meters per unit
inches	0.0254
millimeters	0.001
centimeters	0.01
meters	1.0
mils	2.54×10^{-5}
microns	10^{-6}